Ders 15 Genişletilmiş Notu: Öncelikli Kuyruklar, Heap Soruları ve Huffman Kodlama

Ders 15 Genişletilmiş Notu: Öncelikli Kuyruklar, Heap Soruları ve Huffman Kodlama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Öncelikli Kuyruk Nedir?
- 2. Max ve Min Öncelikli Kuyruk ve Gerçek Hayat Uygulamaları
- 3. Heap İşlemlerinin Öncelikli Kuyruk Gözüyle Tekrarı
- 4. Max Öncelikli Kuyruğun C Kodu
- 5. Soru: Verilen Dizi Bir Heap mi?
- 6. Soru: k'ıncı En Küçük Eleman
- 7. Soru: Min Heap'i Max Heap'e Çevirme
- 8. Huffman Ağacı ile Kayıpsız Veri Sıkıştırma
- Örnek Sorular
- Dersin Toparlaması

1. Öncelikli Kuyruk Nedir?

Hoca bu haftanın konusunun geçen haftaki heap konusuyla çok örtüştüğünü söyledi. Aslında ikisi tek haftada bitebilirdi. Konuyu sindirmek ve vizeden önce aşırı yüklenmemek için iki haftaya yayılmıştı.

Konu öncelikli kuyruk (priority queue) adlı özel bir veri yapısıdır. Adında "kuyruk" geçer, ama önceki haftalarda gördüğümüz ilk giren ilk çıkar (first in first out, FIFO) kuyruğu değildir. Burada elemanların işlenme sırası, girdikleri sıraya göre değil, önceliklerine göre belirlenir.

Hoca iki günlük hayat örneği verdi:

- **Banka sıra makinesi:** Bankaya girince hesap numaranızı ya da kimlik numaranızı girersiniz ve size buna göre bir sıra numarası verilir. Müşteri olanlar daha öne alınabilir.
- **Acil servis:** Gece yarısı bir hastanenin aciline giderseniz kapıda "neyin var?" diye sorulur. Yalnız başı ağrıyan bekletilir. Ağır yaralı hemen içeri alınır. Daha hafif ama acil bir durumu olan ikinci sıraya konur. Acil olmayan bir şikâyetle gelen ise "sıra gelirse bakılır" diyerek sona atılır.

Öncelikli kuyruk, her elemanın bir anahtar (key) değeriyle ilişkilendirildiği bir eleman kümesini tutan veri yapısıdır. Anahtar değeri probleme göre yorumlanır. Küçük sayının önemli olduğu durumda bunu "öncelik sırası" gibi, büyük sayının önemli olduğu durumda da "üstünlük" ya da "güç" gibi okuyabilirsiniz. Elemanlara birer sayısal değer atanır. Bu değerler büyüklük-küçüklük ile karşılaştırılarak elemanların işlenme sırası istenen biçimde düzenlenir.

Öncelikli kuyruk heap ağacı mantığıyla kurulur. Bunun büyük bir avantajı vardır: İlk işlenecek eleman heap'te her zaman köktedir. Dizi gerçekleştiriminde bu, dizinin birinci gözüdür. "Sıradaki gelsin" dendiğinde en öncelikli eleman hiçbir işlem yapmadan $O(1)$ karmaşıklıkta alınır.

Eleman almak aslında kuyruktan eleman çıkarmaktır. Yeni bir müşteri ya da hasta geldiğinde de kuyruğa eklenmesi gerekir. Yeni gelene bir öncelik değeri atanır. Bu eleman sona da düşebilir, araya da girebilir, en öncelikli de olabilir. Bu işlemler için bir yeniden düzenleme gerekir. Heap burada da mantıklı bir çözümdür. Heap tam ikili ağaç (complete binary tree) olduğu için yüksekliği $\log n$ mertebesindedir. Bu yüzden ekleme ve çıkarma işlemlerinin hepsi $O(\log n)$ karmaşıklıktadır.

2. Max ve Min Öncelikli Kuyruk ve Gerçek Hayat Uygulamaları

2.1 Max Öncelikli Kuyruk

Max öncelikli kuyrukta (max priority queue) doğal olarak bir max heap kullanılır. Kökte, yani ilk hizmet alacak elemanda, en yüksek öncelik değeri bulunur. Böyle bir yapının şu işlemleri yapabilmesi gerekir:

- **Ekleme (`insert`):** Kuyruğa bir x elemanını öncelik değeriyle birlikte eklemek.
- **Sorgulama (`maximum`):** En yüksek öncelikli elemanın ne olduğunu sormak. Bu, dizinin birinci gözünü döndürmektir: `return A[1]`.
- **Çıkarma (`extract_max`):** En yüksek öncelikli elemanı kuyruktan çıkarmak.
- **Öncelik artırma (`increase_key`):** Kuyruktaki bir elemanın önceliğini herhangi bir anda yükseltmek.

Hoca `increase_key` için yine banka örneğini kullandı. Sıra makinesinden düğmeye basıp 333 numarayı aldınız. Öncelikli kuyrukta bir yerlerdesiniz. Müşteri numarasını girip daha öncelikli numara alanlar var. Siz de beklemekten sıkıldınız ve "ben de müşteriyim" deyip müşteri numaranızı girdiniz. Size 172 gibi daha öncelikli bir numara verildi. Bir anda pek çok kişinin önüne geçtiniz.

Bunu çağrılma sırası yerine değer olarak düşünmek gerekir. Max heap'te büyük değer öncelikli olduğuna göre, müşteri değeriniz eskiden 50 iken müşteri numaranızı girince 80'e, 90'a çıktı. Heap'te daha yukarı bir yere geçtiniz. Belki o anda kuyruktaki herkesten büyük bir değer aldınız ve köke çıktınız; sıradaki hizmeti siz alırsınız. Genel olarak bir elemanın önceliği $k \geq x$ koşulunu sağlayan bir k değerine çıkarılabilir; burada x elemanın mevcut değeridir.

Uygulama: İşlem zamanlayıcı (process scheduler). Çok görevli (multitasking) bir işletim sisteminde tek çekirdek varsa aynı anda yalnız bir süreç (process) çalışır. Ama her sürece sırayla bir çalışma zamanı dilimi verilerek süreçlerin eş zamanlı çalışıyormuş gibi görünmesi, yani çok programlı bir ortam sağlanır. Zamanlayıcının hızlı olması gerekir: İşlemciye sıradaki öncelikli süreci hemen verebilmelidir. Bu yüzden çıkarma işlemi en az karmaşıklıkla yapılmalıdır.

Her sürece bir öncelik değeri atanır ve süreçler bir öncelikli kuyrukta tutulur. Bir süreç biterse ya da ona ayrılan zaman dilimi dolarsa, zamanlayıcı onu yeni bir öncelik değeriyle kuyruğa geri koyar ve sıradaki en yüksek öncelikli süreci `extract_max` ile devreye alır. n süreç varken:

- En öncelikli süreci sorgulamak $O(1)$'dir.
- `extract_max` $O(\log n)$'dir.
- Kullanıcı yeni bir süreç başlatırsa `insert` $O(\log n)$'dir.
- Kullanıcı bir sürecin önceliğini artırır ya da azaltırsa bu da en fazla $O(\log n)$'dir.

2.2 Min Öncelikli Kuyruk

Bunun tersi min öncelikli kuyruktur (min priority queue). Burada min heap kullanılır. `insert` değişmez. Sorgulama `maximum` yerine `minimum` olur. `extract_max` yerine en düşük öncelik değerine sahip elemanı alan `extract_min` kullanılır. `increase_key`'in tersi olarak da `decrease_key` tanımlanır. Anahtar değeri küçük olan önceliklidir.

Uygulama: Olay güdümlü benzetim (event-driven simulation). Hoca banka örneğinin yanında benzetim programlarının (simulator) da böyle çalıştığını söyledi. Diyelim ki bilimsel bir deneyi benzetiyorsunuz. Çeşitli tepkimeler ve etkileşimler sonucunda olaylar oluşuyor. Her olayın bir oluşma zamanı var. Bu zamanlar sizin anahtarlarınızdır. Bir zaman çizelgesindeki olayları benzetirken önce olanı önce, sonra olanı sonra benzetmek gerekir.

Üstelik bir olay yeni olaylar doğurabilir. Yeni olaylar en sona değil, arada bir yere düşebilir. Belki hemen bu olaydan sonra benzetilmesi gerekir. Bu yüzden benzetici bir min heap kurar ve onu min öncelikli kuyruk olarak kullanır:

- `extract_min` yaptığında sıradaki olayı seçmiş olur.
- Yeni bir olay oluştuğunda onu gerçekleşme zamanıyla birlikte `insert` eder.
- Bir olayın öne alınması gerekirse `decrease_key` ile onu şimdiki benzetim zamanına yakın bir sıraya çeker.

Özetle bir min heap ya da max heap kurulur. Heap düzeninde kullanılan sayılar anahtar değerlerdir. Ya yüksekler ya düşükler öncelikli sayılarak bir öncelik sırası mekanizması işletilir.

3. Heap İşlemlerinin Öncelikli Kuyruk Gözüyle Tekrarı

3.1 En Öncelikli Elemanı Sorgulamak

Max ya da min sorgulamak, en yüksek ya da en düşük öncelikli elemanın hangisi olduğunu sormaktır. Bu eleman heap'in köküdür: `return A[1]`. Dizinin birinci gözünü döndürmek yeterlidir; karmaşıklık $O(1)$'dir.

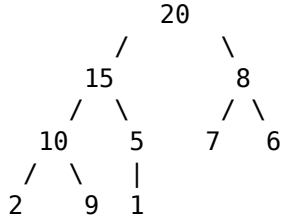
3.2 `extract_max`

Adımlar şunlardır:

1. Heap'in boş olmadığından emin ol.
2. Kökteki değeri `max` adlı bir değişkene kopyala.
3. Son elemanı bilerek ve isteyerek köke getir.
4. Eleman sayısını bir azalt ve `max_heapify` fonksiyonunu kökten başlayarak $n - 1$ eleman için çağır.
5. `max` değerini döndür.

Döndürülen değer, heapify öncesinde kökte olan, yani en yüksek öncelikli elemandır. Örneğin işlem zamanlayıcısına "al bu süreci" diye verilir. Heapify en kötü durumda ağacın yüksekliği kadar adım attığı için `extract_max` $O(\log n)$'dir.

Hoca geçen haftaki örneği tekrar gösterdi. Heap `20, 15, 8, 10, 5, 7, 6, 2, 9, 1` dizisidir:



- Kök 20 çıkarılır. Son eleman 1 köke getirilir ve eleman sayısı 9'a düşer.
- 1'in çocukları 15 ve 8'dir. En büyüğü 15'tir. 15 yukarı çıkar, 1 aşağı iner.
- 1'in yeni çocukları 10 ve 5'tir. 10 yukarı çıkar, 1 aşağı iner.
- 1'in yeni çocukları 2 ve 9'dur. 9 yukarı çıkar, 1 aşağı iner.
- Sonuç: `15, 10, 8, 9, 5, 7, 6, 2, 1`. Bir daha `extract_max` yapılırsa 15 çekilir.

3.3 Fibonacci Heap Hakkında Bir Not

Hoca bazı uygulamalarda çıkarmanın $O(\log n)$ sürmesinin yeterli olmayabileceğini söyledi. O zaman öncelikli kuyruktaki ikili heap yerine başka heap türleri kullanılabilir. Bu ders temelleri verdiği için bunlar anlatılmadı. Örnek olarak Fibonacci heap'i verdi.

> **Ders sonrası not:** Hoca Fibonacci heap'te çıkarmanın $O(1)$, ekleme ve değiştirmenin $O(\log n)$ olduğunu söyledi. Durum tersidir. Fibonacci heap'te `minimum`, `insert` ve `decrease_key` amortize $O(1)$; `extract_min` ise amortize $O(\log n)$'dir. Yani Fibonacci heap ekleme ve anahtar güncellemeyi hızlandırır, çıkarmayı değil.

3.4 `increase_key`

Hoca bu kısmı slayttaki animasyon çalışmadığı için çizerek anlattı. Bir düğümün değeri 4 iken 15 yapılıyor. Değer büyüdüğü için düğümün yukarıya doğru doğru yerde olup olmadığı kontrol edilir; ebeveyniyle karşılaştırılır:

- 15'in ebeveyni 8'dir. 15 daha büyüktür; yer değiştirirler. Ebeveynin öbür çocuğundan büyük olduğu zaten bilindiği için onunla ayrıca karşılaştırmaya gerek yoktur.
- 15'in yeni ebeveyni 14'tür. 14 daha küçüktür; yer değiştirirler.
- 15'in yeni ebeveyni 16'dır. 16 daha büyüktür; işlem durur.

Hocanın yorumu şuydu: Eskiden 4 "kaplan gücünde" bir müşteri vardı. Sıkıldı, kredi kartıyla sıra numarası aldı ve artık 15 kaplan gücünde. Öncelikli kuyrukta ikinci sıraya çıktı. 16 gücündeki müşteri hizmet alıp gidince 15 köke gelecek. Çok aşağıdayken bir anda ikinci sırada hizmet alacak kişi oldu.

3.5 `insert` ve Neden $-\infty$?

Yeni eleman eklemek için `increase_key` kullanılır:

1. Heap'in eleman sayısını bir artır.
2. Yeni açılan son göze $-\infty$ yaz.
3. Bu gözün değerini `increase_key` ile istenen anahtara çıkar; eleman yukarıda olması gereken yere taşınır.

Hoca neden $-\infty$ ile başlandığını sordu ve cevapladı: heap düzenini bozmamak için. Diyelim ki yeni gözü açtınız ve tam o anda, değeri yerine taşımadan önce, kuyruktan bir çıkarma işlemi gerçekleşti. Eğer o göze doğrudan bir değer yazmış olsaydınız heap düzeni bozuk kalacaktı. $-\infty$ hiçbir ebeveynden büyük olamayacağı için yapı her an geçerli bir heap'tir. Sonra `increase_key` ile eleman gerçek yerine çıkarılır.

4. Max Öncelikli Kuyruğun C Kodu

Hoca hep akış ve sözde kod (pseudocode) üzerinden anlattıklarını bu kez C koduyla da göstermek istedi. Slayta sığacak kadar basitleştirilmiş, max heap kullanan bir max öncelikli kuyruk kütüphanesidir. "Siz çok daha iyisini yazarsınız" dedi.

4.1 Sabitler ve `swap`

Sonsuz için normalde `limits.h` içindeki `INT_MAX` kullanmak daha doğrudur. Burada basitlik için `INF` adlı bir sabit 100000 olarak tanımlanmıştır. Hoca "100000 yeterince büyük mü? Olmayabilir, idare edin" dedi. Heap başta boştur, yani `heap_size` 0'dır. Ağaç en fazla 20 elemanlık bir dizi olarak tanımlanmıştır.

Eleman değiştirme (swap) her yerde gerekir. İki tamsayının adresi alınır, bir geçici değişken üzerinden değerler değiştirilir.

4.2 Çocuk ve Ebeveyn Fonksiyonları

Heapify ya da `increase_key` yaparken "çocuklarım kim, ebeveynim kim?" sorularına cevap gerekir. Dizi gerçekleştiriminde i indisindeki düğümün sol çocuğu $2i$'de, sağ çocuğu $2i+1$'dedir; ebeveyni $\lfloor i/2 \rfloor$'dedir.

Bu fonksiyonlar indisi döndürmeden önce sınır kontrolü yapar. İndis 1'in altına düşmemeli ve dizinin boyutunu aşmamalıdır. Aksi hâlde heap'in dışına taşılır ve bellekte yanlış bir yer okunabilir. Aralık dışındaysa "böyle bir çocuk yok" anlamında -1 döndürülür. Hocanın örneği: 5 elemanlı bir heap'te 3. düğümün sol çocuğu 6. göz olurdu. 6 eleman sayısını aştığı için çocuk yoktur.

Ebeveyn fonksiyonu da $i/2$'yi döndürür. Tamsayı bölme olduğu için hem $2i$'den hem $2i+1$'den i'ye gidilir. Kökteyken (`index` 1 iken) "index 1'den büyük mü?" kontrolü başarısız olur ve -1 döner: "Sen zaten köksün, ebeveynin olamaz."

4.3 `max_heapify`, `build_max_heap` ve Kuyruk İşlemleri

`max_heapify` önce sol ve sağ çocuğun indisini alır. `largest` başta düğümün kendisidir. Sol çocuk daha büyükse `largest` odur. Sonra sağ çocuk `largest` ile karşılaştırılır ve daha büyükse `largest` güncellenir. Sonunda `largest` çocuklardan biriyse bu alt ağaç heap değildir. O zaman düğüm ile `largest` yer değiştirir ve `largest`'tan başlayarak aşağı doğru yine `max_heapify` çağrılır. Hocaya göre bu kodun sözde koddan hiçbir farkı yoktur; yalnız C diline çevrilmiştir.

`build_max_heap` aşağıdan yukarı yaklaşımı (bottom-up approach) uygular: `heap_size / 2`'den başlayıp 1'e kadar azalarak her ebeveyne `max_heapify` uygulanır.

`extract_max` birinci gözdeki değeri alır, `A[1]`'e son elemanı yükler, `heap_size`'ı bir azaltır ve birinci gözden `max_heapify` yapar. `increase_key` verilen indise yeni anahtarı yazar; değer ebeveyninden büyük olduğu sürece yer değiştirip ebeveyne geçer. Max öncelikli kuyrukta `decrease_key`, elemanın önceliğinin düştüğü anlamına gelir: Değer yazılır ve çocuklarla karşılaştırmak için `max_heapify` çağrılır; özyineleme düğümü inmesi gereken seviyeye kadar indirir. `insert` boyutu artırır, yeni göze `-INF` yazar ve `increase_key` çağırır.

```c
#include <stdio.h>

#define INF 100000          /* basitlik için; normalde limits.h içindeki INT_MAX */
#define tree_array_size 20

int heap_size = 0;

void swap(int *a, int *b) {
    int t = *a;
    *a = *b;
    *b = t;
}

int get_right_child(int A[], int index) {
    if (((2 * index) + 1 < tree_array_size) && (index >= 1))
        return (2 * index) + 1;
    return -1;
}

int get_left_child(int A[], int index) {
    if (((2 * index) < tree_array_size) && (index >= 1))
        return 2 * index;
    return -1;
}

int get_parent(int A[], int index) {
    if ((index > 1) && (index < tree_array_size))
        return index / 2;
    return -1;
}

void max_heapify(int A[], int index) {
    int left_child_index = get_left_child(A, index);
    int right_child_index = get_right_child(A, index);
    int largest = index;

    if ((left_child_index <= heap_size) && (left_child_index > 0))
        if (A[left_child_index] > A[largest])
            largest = left_child_index;

    if ((right_child_index <= heap_size) && (right_child_index > 0))
        if (A[right_child_index] > A[largest])
            largest = right_child_index;
```

```
    if (largest != index) {     /* düğüm en büyük değilse burası heap değildir */
        swap(&A[index], &A[largest]);
        max_heapify(A, largest);
    }
}

void build_max_heap(int A[]) {
    int i;
    for (i = heap_size / 2; i >= 1; i--)
        max_heapify(A, i);
}

int maximum(int A[]) {
    return A[1];
}

int extract_max(int A[]) {
    int maxm = A[1];
    A[1] = A[heap_size];
    heap_size--;
    max_heapify(A, 1);
    return maxm;
}

void increase_key(int A[], int index, int key) {
    A[index] = key;
    while ((index > 1) && (A[get_parent(A, index)] < A[index])) {
        swap(&A[index], &A[get_parent(A, index)]);
        index = get_parent(A, index);
    }
}

void decrease_key(int A[], int index, int key) {
    A[index] = key;
    max_heapify(A, index);
}

void insert(int A[], int key) {
    heap_size++;
    A[heap_size] = -1 * INF;
    increase_key(A, heap_size, key);
}

void print_heap(int A[]) {
    int i;
    for (i = 1; i <= heap_size; i++)
        printf("%d ", A[i]);
    printf("\n");
}

int main(void) {
    int A[tree_array_size];
    int values[] = {20, 15, 8, 10, 5, 7, 6, 2, 9, 1};
    int i;

    for (i = 0; i < 10; i++)
        insert(A, values[i]);
    print_heap(A);                  /* 20 15 8 10 5 7 6 2 9 1 */

    increase_key(A, 5, 22);
```

```
    print_heap(A);                  /* 22 20 8 10 15 7 6 2 9 1 */

    decrease_key(A, 1, 13);
    print_heap(A);                  /* 20 15 8 10 13 7 6 2 9 1 */

    printf("%d\n", extract_max(A)); /* 20 */
    print_heap(A);                  /* 15 13 8 10 1 7 6 2 9 */

    while (heap_size > 0)
        printf("%d ", extract_max(A));
    printf("\n");                   /* 15 13 10 9 8 7 6 2 1 */
    return 0;
}
```

- Çocuk/ebeveyn fonksiyonları, `max_heapify`, `increase_key` ve `decrease_key` slayttaki biçimiyle verildi. Sabitler, `swap`, `build_max_heap`, `extract_max`, `insert`, `print_heap` ve `main` hocanın anlatımından kuruldu; slayttaki yazımdan küçük farklar olabilir.
- Kod `gcc -Wall` ile uyarısız derlenir. Yorumlardaki çıktılar programın gerçek çıktısıdır.

  Ders sonrası not: Slayttaki `extract_max` heap boşken kontrol yapmıyor. Gerçek kullanımda `heap_size < 1` ise hata döndürülmelidir. Hoca da sözlü anlatımda ilk adımın "heap'in boş olmamasını sağlamak" olduğunu söylemişti.

4.4 Sürücü Kodunun İzlenmesi

Heap'i yazdırmak için yalnızca bir döngüyle dizi yazdırılır. Sürücü kod (driver code) şu adımları yapar:

Ekleme. 20, 15, 8, 10, 5, 7, 6, 2, 9, 1 sırayla eklenir. Önce 20 köke konur. 15 ikinci göze gelir; ebeveyninden küçük olduğu için yerinde kalır. 8 üçüncü göze gelir. 10 dördüncü göze gelir, 15'ten küçüktür, kalır. 5, 7, 6, 2, 9 ve 1 de geldikleri gözde kalır. Hoca "zaten ben de güzel yazmışım, hiçbir şeyin yeri değişmiyor" dedi. Her eklemede eleman ebeveyniyle karşılaştırılır ve daha küçük olduğu görülür. Sonuç bir max heap'tir: `20, 15, 8, 10, 5, 7, 6, 2, 9, 1`.

`increase_key(A, 5, 22).` 5 değerine sahip eleman "ben 22 olmak istiyorum" der.

- 5. göz 22 olur. Ebeveyni 15'tir (2. göz). 15 aşağı, 22 yukarı.
- 22'nin yeni ebeveyni 20'dir. 20 aşağı, 22 yukarı.
- Sonuç: `22, 20, 8, 10, 15, 7, 6, 2, 9, 1`.

`decrease_key(A, 1, 13).` Birinci gözdeki 22, 13'e indirilir.

- 13'ün çocukları 20 ve 8'dir. 20 yukarı çıkar, 13 iner.
- 13'ün yeni çocukları 10 ve 15'tir. 15 yukarı çıkar, 13 iner.
- 13'ün tek çocuğu 1'dir. 13 daha büyüktür; doğru yerdedir.
- Sonuç: `20, 15, 8, 10, 13, 7, 6, 2, 9, 1`.

`extract_max.` 20 alınır. 1 köke konur ve kökten heapify yapılır: 15 çıkar, sonra 13 çıkar. Sonuç: `15, 13, 8, 10, 1, 7, 6, 2, 9`. Ardından "extract yap, yazdır" döngüsüyle kalan elemanlar büyükten küçüğe çekilebilir.

Hoca öncelikli kuyrukla ilgili teorik anlatacaklarının bu kadar olduğunu söyledi ve heap ile ilgili birkaç soruyu birlikte düşünmeye geçti.

5. Soru: Verilen Dizi Bir Heap mi?

Hocanın ilk sorusu şuydu: Kullanıcıdan 1'den n'e kadar indisli bir dizi okundu; örneğin 7 değer. Bu dizi bir heap ağacı mıdır, nasıl anlarız?

> **Hocanın vurgusu:** "Güzel sınav sorusu olurmuş bundan galiba."

Bir öğrenci önce sol ve sağ tarafın uzunluğuna bakmayı önerdi, sonra soruyu bağlı liste gibi düşündüğünü fark etti. Ardından iki ayrı fonksiyon önerdi: Biri ağacın uzunluğunu bulacak, öbürü her çocuğun ebeveyninden küçük olduğunu kontrol edecek. Birinci fonksiyonu kökten başlayıp 2 ve 3'e, sonra $4, 5$ ve $6, 7$'ye bakan özyinelemeli bir fonksiyon olarak anlattı; iki tarafın

uzunluk farkı 0 ya da 1 ise ağaç heap olabilirdi. Hoca 1, 2, 3, 4 düğümlü bir ağaç çizerek bu fikri sorguladı. Öğrenci "kodunu yazsam daha hızlı anlatırım" deyince hoca "sürat felakettir, düşünerek yaz" dedi.

Hoca soruyu "heap olmanın şartı nedir?" diye geri çevirdi. Geçen hafta heap'i heap yapan iki şey olduğunu söylemişti:

1. **Tam ikili ağaç olmak.** "Tamamen dolu" değil, tam ikili ağaç (complete binary tree): Bütün seviyeler tam dolu, son seviye soldan doludur. Buna `is_complete_binary_tree` denebilir.
2. **Ebeveyn kontrolü.** Max heap için her ebeveyn çocuklarından büyük ya da eşit olmalıdır. Buna `parent_control` denebilir.

Problem iki aşamada çözülür.

Birinci aşama: Tam ikili ağaç mı? Bu birkaç şekilde kontrol edilebilir:

- Özyinelemeli bir kontrol yazılabilir.
- Seviye sayısı hesaplanabilir. Dizinin uzunluğu 7 ise ağaç 3 seviyelidir. İlk iki seviyede toplam 3 düğüm vardır; 1., 2. ve 3. gözler dolu mu diye bakılır. Sonra 4. gözden başlayıp bir yere kadar dolu gidip gitmediği, ondan sonra n'e kadar boşluk olmadığı kontrol edilir.
- Sondan başlayarak her ebeveynin 0, 1 ya da 2 çocuğu olup olmadığına bakılabilir.

Buradan mutlaka `true` dönmelidir. Tam ikili ağaç değilse zaten heap olamaz.

İkinci aşama: Ebeveynler çocuklarından büyük mü? Ebeveynler dizinin ilk $\lfloor n/2 \rfloor$ elemanındadır. Çocukları $2i$ ve $2i+1$'dedir. $\lfloor n/2 \rfloor$'den 1'e kadar inen (`downto`) bir döngüyle her i için $A[i] \geq A[2i]$ ve (varsa) $A[i] \geq A[2i+1]$ kontrol edilir. Bütün ebeveynler çocuklarından büyükse `return true` döndürülür.

Hoca bunun tek bir döngüyle de yapılabileceğini, ama biraz daha karışık olacağını söyledi ve bunu öğrencilere bıraktı.

> **Ek bilgi (derste anlatılmadı):** 1'den başlayan dizi gerçekleştiriminde n elemanlı bir dizi, $1..n$ gözleri arasında boşluk yoksa her zaman bir tam ikili ağacı temsil eder. Bu yüzden diziyle verilen bir heap'te tamlık kontrolü, aslında ara gözlerde "boş" işaretli bir değer olup olmadığını kontrol etmeye dönüşür. Tek döngülü sürümde, boş göz kontrolü ile ebeveyn-çocuk karşılaştırması aynı döngüde yapılabilir.

6. Soru: k'ıncı En Küçük Eleman

Hocanın ikinci sorusu, "herkesin bir şekilde duymuş olabileceği" bilindik bir sorudur. n elemanlı, 1'den başlayan bir A dizisi veriliyor. Dizi sıralı değil, karışıktır. Basit olsun diye tekrar eden eleman yoktur; n farklı (distinct) elemandan oluşur. $k \leq n$ olmak üzere dizinin k'ıncı en küçük elemanı nasıl bulunur?

> **Hocanın vurgusu:** Bunu bir vize sorusu olarak düşünün: "Tekrar etmeyen elemanlara sahip, sırasız, n elemanlı bir dizinin k'ıncı en küçük elemanını bulan algoritmayı tasarlayınız, akışını çiziniz." Sorulan soruların genelde herkesin aklına gelen düz bir çözüm yolu olur. Bir de ince (tricky) noktaları olur; örneğin iç içe iki döngü yerine bir kontrol ya da ek bir yapı kullanarak farklı çözebiliyor musunuz? Asıl ölçülen budur.

6.1 Düz Çözüm: Sırala

Bir öğrencinin ilk önerisi buydu: Diziyi sırala, k'ıncı gözü döndür. Akış diyagramında kolayca gösterilebilecek sıralamalar kabarcık (bubble), seçmeli (selection) ve eklemeli (insertion) sıralamadır. Hoca büyük ihtimalle seçmeli sıralama yazılacağını söyledi: iç içe iki `for` ve iç döngünün dışında minimumu güncelleme. Bu n^2 artı sabit, yani $O(n^2)$ zaman gerektirir.

"Bunu $O(n \log n)$'de çözün" denseydi, geçen hafta görülen yığın sıralaması (heap sort) ile sıralayıp yine k'ıncı gözü döndürmek mümkündü. Ama yine de sıralamak gerekiyordu. Hoca şunu sordu: Bir veri yapısının gücünü kullanarak daha iyi bir çözüm geliştirilebilir mi? Bağlı liste mi, matris mi, yığın (stack) mı, kuyruk (queue) mu, ağaç mı, heap mi?

6.2 Öğrencinin Kuyruk Önerisi

Bir öğrenci kuyruk kullanmayı önerdi: Boyutu k olan bir kuyruğa dizinin ilk k elemanı atılır. Kuyruk dolunca, yeni gelen eleman kuyruktaki en büyük değerden küçükse o değer çıkarılır ve yeni eleman alınır. Hoca kuyruğun bir uçtan girilip yalnız öbür uçtan çıkılan bir yapı olduğunu hatırlattı; ortadaki en büyük elemanı çıkaramazsınız. Öğrenci öneriyi heap ile düzeltti: Yeni değer heap'in başındaki değerden küçükse o değeri çıkarıp yeni elemanı almak. Hoca bu fikri az sonra formalize edeceğini söyledi.

6.3 Min Heap ile Çözüm

Hoca soruyu basitleştirdi: Benden k'ıncı değil de birinci en küçük eleman istenseydi ne yapardım? En küçük elemanı "tak diye", hiç yorulmadan verebilen veri yapısı heap'tir. O hâlde n elemandan bir min heap kurulur. Bu, dizideki en küçük elemanın kesinlikle `A[1]`'de olacağını garanti eder. Birinci en küçük için problem çözülmüş olur.

k'ıncı en küçük için `extract_min` tekrarlanır. Birinci çıkarma $\log n$, ikinci $2\log n$, ... k'ıncı çıkarma toplam $k \log n$ maliyete ulaşır. k'ıncı kez çıkarılan değer aranan değerdir. Hoca heap kurmanın maliyetini n eleman ve her biri için en fazla $\log n$ işlemle $O(n \log n)$ olarak verdi. Toplam:

$$O(n \log n) + O(k \log n) = O\big((n + k) \log n\big)$$

Bir öğrencinin olası itirazını hoca kendisi sordu: "Ne kazandık? Heap sort da $n \log n$ idi." Cevabı şuydu: Heap sort'taki $n \log n$'in de yanında terimler vardı, sabit oldukları için atılmıştı. Burada k, n'den çok küçükse ($k \ll n$) önemsizleşir. Hocanın örneği: 85 milyon kişi içinden en yaşlı ya da en genç dördüncü ya da yüzüncü kişiyi bulmak. En kötü durumda $k = n$ olur; $n \log n + n \log n = 2n \log n$ olur, katsayı 2 atılır ve yine $O(n \log n)$ mertebesi kalır.

> **Ders sonrası not:** Hoca min heap kurmanın maliyetini $O(n \log n)$ olarak verdi. Heap, n elemanı tek tek eklemek yerine `build_heap` ile aşağıdan yukarı kurulursa maliyet $O(n)$'dir. Bu durumda min heap çözümünün toplamı $O(n + k \log n)$ olur. Hocanın verdiği $O((n + k) \log n)$ bir üst sınır olarak yanlış değildir, ama sıkı değildir.

6.4 k Elemanlı Max Heap ile Daha İyi Çözüm

Hoca "ben bunu max heap ile çözmek istiyorum, garip düşünmeyi seviyorum" diyenler için öğrencinin fikrini biraz iyileştirerek şu çözümü verdi:

1. Dizinin ilk k elemanını alıp bir max heap kur (onları max heap'e `insert` et).
2. Kalan $n - k$ elemanın her birini max heap'in kökündeki elemanla karşılaştır.
3. Eleman kökten küçükse kökü sök, yeni elemanı k elemanlı heap'e ekle ve heapify et.
4. Bütün elemanlar gezildiğinde heap'te dizinin en küçük k elemanı kalır. Max heap olduğu için bunların en büyüğü, yani k'ıncı en küçük eleman, köktedir. `A[1]` doğrudan okunur, $O(1)$.

Hoca bunu $k = 4$ için küçük bir max heap çizerek tekrarladı: İlk 4 elemandan heap kurulur; 5. elemandan n'e kadar her eleman kökle karşılaştırılır. Heaptekilerden küçük bir değer gelirse "en büyük değeri dışarı at, yerine küçük değeri koy, heapify et" denir. Böylece heap her zaman k elemanlıdır.

Karmaşıklık:

- İlk k elemandan heap kurmak: $O(k \log k)$.
- Kalan $n - k$ elemanın her biri için en fazla bir yeniden düzenleme: $(n - k) \log k$.
- Toplam: $k \log k + (n - k) \log k = n \log k$. Hoca "$k \log k$ ile $-k \log k$ gitti, yalnız $n \log k$ kaldı" dedi.
- Ek alan: heap kadar, yani $O(k)$.

$$O(k \log k + (n - k) \log k) = O(n \log k)$$

k yeterince küçük olduğu için bu daha iyidir. n 80 milyon, k 100 ya da 3, 5, 10, 1000 gibi düşünülürse, n çok küçük bir katsayıyla çarpılmış olur. Çözüm doğrusala (linear) çok yakındır.

> **Ders sonrası not:** Hoca max heap çözümünü özetlerken bir yerde "minimum 4 elemanı tut" dedi ve hemen "minimum k eleman" diye düzeltti. 4 yalnız çizdiği örnekteki k değeridir; genel hâli "dizideki en küçük k eleman"dır.

6.5 Hangi Veri Yapısı Kötü Çalışır?

Soru: Bu tür sorularda en dezavantajlı, yani en büyük karmaşıklığı verecek veri yapısını nasıl belirleriz? Bir veri yapısının hangi problemlerde avantajlı olduğunu söyleyebiliyoruz; hangilerinde en kötü sonucu vereceğini de söyleyebilir miyiz?

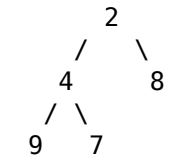
Cevap: Hazır bir kural yoktur; tamamen probleme bağlıdır. Önce problemi bilmek gerekir. Hoca önceki haftalardan örnek verdi: Hep sona ekleme ve sondan çıkarma yapılıyorsa dizi kullanılır, bağlı liste gerekmez. Araya ekleme ve çıkarma yapılıyorsa dizi çok maliyetli olur, bağlı liste işi kolaylaştırır. Bağlı listede arama doğrusaldır ve maliyetlidir; arama sıksa ikili arama ağacı (binary search tree) tutmak düşünülebilir. Bunlar problemin doğasına hangi yapının daha iyi hizmet edeceğini düşünerek ortaya çıkar.

> **Hocanın vurgusu:** k'ıncı en küçük probleminde ilk akla gelen min heap'tir, çünkü min heap bu düzeni zaten sağlar. Ama tersten düşünüp küçük bir max heap kullanınca karmaşıklık $n \log k$'ya iner. Veri yapısını seçmek bir adımdır; o veri yapısının varyantlarını düşünmek ise ikinci, daha da iyileştiren adımdır.

7. Soru: Min Heap'i Max Heap'e Çevirme

Hocanın üçüncü sorusu: Bir min heap verildi (aynı şey bir öncelikli kuyruk için de sorulabilir). Bunu bir max heap'e nasıl dönüştürürüm?

Bir süre cevap gelmedi. Bir öğrenci min heap'in son elemanını yeni kök olarak almayı önerdi: "Min heap'in son elemanı max heap'in köküdür." Hoca son elemanın en büyük olmak zorunda olmadığını gösterdi. Önce `8, 6, 4, ...` diye bir ağaç çizmeye başladı, bunun max heap olduğunu fark edip tersten bir min heap yazdı: `2, 4, 8, 9, 7`.



Son eleman 7'dir, ama en büyük eleman 9'dur. 7'yi köke koymak yapıyı "pat diye yamultur". Öğrenci kökü başlangıç olarak almanın en makul yol olduğunu, sonra çocukları uygun mu diye tek tek heapify edeceğini söyledi.

Hoca bu "tek tek heapify" fikrini geliştirdi: Bunu yalnız çocuğu olan düğümler için yapmak yeterli olmaz mı? Evet yeterlidir. Çocuğu olmayan düğüm tek elemanlıdır; kendi içinde hem min heap hem max heap'tir. O hâlde heap kurar gibi $\lfloor n/2 \rfloor$'den 1'e doğru bütün ebeveynler işlenir. Min heap'ten max heap'e geçmek için `max_heapify`, max heap'ten min heap'e geçmek için `min_heapify` kullanılır. `max_heapify` özyinelemeli olarak aşağı indiği için sorun çıkmaz. Kökü ya da son elemanı özel olarak seçmeye gerek yoktur.

Hocanın çizdiği min heap üzerinde bu işlem derste adım adım yapılmadı; sonuç bu bölümün sonundaki Örnek Sorular'da verilmiştir.

8. Huffman Ağacı ile Kayıpsız Veri Sıkıştırma

8.1 Amaç ve Sezgi

Hoca bazı öğrencilerin bu konuyu ayrık matematikte görmüş olabileceğini, ama veri yapısıyla birlikte anlatmanın faydalı olduğunu söyledi. Huffman ağacı, günlük bilgisayar işlerinde farkında olmadan varyantları kullanılan bir çözümdür. Amacı veri sıkıştırmaktır. Hocaya göre

özel bir tarafı da yoktur: Yalnız bir öncelikli kuyruk oluşturulur. Elinizdeki dosyanın karakterleri kullanılarak bir öncelikli kuyruk kurulur. Kurulan ağaçta karaktere giden yol, o karakterin yeni kodu olur. Bütün mantık buna dayanır.

Her karakteri 1 byte, yani 8 bitle kodlamak yerine çok sık kullanılanları daha az bitle kodlamak toplamda bit, yani alan tasarrufu sağlar. Aynı veri başka bir gösterimle daha az yer kaplıyorsa veri sıkıştırılmış olur. Aynı veri geri döndürülerek tekrar elde edilebiliyorsa hiçbir kayıp yoktur; veri kayıpsız (lossless) sıkıştırılmıştır. Yöntemi Huffman önermiştir; adı bu yüzden Huffman ağacıdır.

Karakter derken roman karakteri değil, kelimeleri oluşturan harfler, noktalama işaretleri ve rakamlar kastedilir. Doğal dillerde bazı harfler çok, bazıları az kullanılır. Hocaya göre Türkçede en çok kullanılan ünlüler A ve E'dir, daha az olarak İ gelir; en az kullanılanlar Ü, Ö gibi harflerdir. Ünsüzlerde K, L, M, N çok; Z, J, Ç az kullanılır, J ise çok daha azdır.

Hocanın sayısal sezgisi şuydu: Bir metnin karakterlerinin yarısı A olsun. A'yı 8 bit yerine 3 bitle kodlarsanız her A'da 5 bit kazanırsınız. Metin 1000 karakterse 500 A için $500 \times 5 = 2500$ bit tasarruf edilir. Tabii kodun geri çözülebilmesi, yani oradaki karakterin A mı J mi olduğunun anlaşılabilmesi gerekir.

Buradan kural çıkar: En sık kullanılan karaktere mümkün olan en kısa kod verilmelidir; yani ağaçta mümkün olduğunca yukarıda tutulmalıdır. En az kullanılan karaktere daha uzun kod verilebilir. En kötü durum bütün karakterlerin eşit sıklıkta olmasıdır; o zaman sabit uzunluklu koddan bir kazanç olmaz. Hocaya göre doğal metinlerde bu beklenmez. Sıklıkları eşit olan karakterler de doğal olarak aynı uzunlukta kod alır.

8.2 Neden Min Öncelikli Kuyruk?

Sık karakterin sonuç ağacında köke yakın olması istenir. Bunun için kurma sırasında sık karakterler en sona bırakılmalıdır. Bu yüzden anahtarı karakterin kullanım sıklığı (frekansı) olan bir min öncelikli kuyruk kurulur. En az rastlanan karakter en küçük frekansa sahip olduğu için kökte durur ve önce çekilip işlenir; sonuç ağacında en diplerde yer alır. En sık karakterler en son işlenir; sonuç ağacında köke en yakın yerde durur ve en kısa kodu alır.

> **Ders sonrası not:** Hoca girişte "öncelikli kuyruk aslında bir heap ağacı; heap ağacında her düğümün durduğu yere giden yol karakterin kodu olur" dedi. Kodlar heap'teki konumlardan değil, öncelikli kuyruk yardımıyla kurulan ayrı bir ikili ağaçtan (Huffman ağacı) okunur. Heap yalnız her adımda en küçük iki frekansı hızlıca bulmak için kullanılır. Hocanın sonraki adım adım anlatımı da bu doğru işleyişe uyar.

8.3 Algoritma

1. **Frekansları say.** Basit olsun diye yalnız küçük harfler (a-z) için bir sayaç (counter) dizisi tutulur; istenirse rakamlar, noktalama ve bütün ASCII tablosuna genişletilebilir. Hepsi 0 ile başlar. Metin okunur; gelen her karakterin sayacı bir artırılır.
2. **Min öncelikli kuyruk kur.** Frekans dizisi, frekansları anahtar olan bir min öncelikli kuyruğa dönüştürülür.
3. **İki minimumu çek.** Kuyruktan iki kez `extract_min` yapılır.
4. **Birleştir.** İki düğümün frekansları toplanır. Bu toplamla bir ara düğüm (internal node) oluşturulur. Bu bir karakter düğümü değil, bileşke bir düğümdür. Çekilen iki düğüm onun sol ve sağ çocuğu olur.
5. **Geri ekle.** Ara düğüm kuyruğa `insert` edilir.
6. Kuyrukta tek düğüm kalana kadar 3-5. adımlar tekrarlanır. Kalan düğüm Huffman ağacının köküdür.

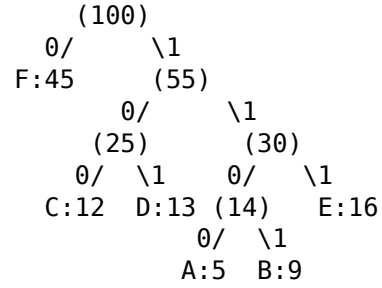
Düğüm yapısında bir karakter alanı, bir frekans alanı, bir sol (`left`) ve bir sağ (`right`) gösterici bulunur. Yaprakta göstericiler `NULL`'dur. Hoca ara düğümleri yuvarlak çizdi, ama gerçekte aynı düğüm yapısı kullanılır: `left` birinci düğümün adresini, `right` ikinci düğümün adresini gösterir; frekans ikisinin toplamıdır; karakter alanına `*` gibi özel bir işaret konur. Ara düğümün içinden alt düğümlere `left` ve `right` adresleriyle erişilebilir.

8.4 Örnek: Altı Karakterli Girdi

Girdide yalnız 6 sembol vardır: A, B, C, D, E, F. Sayınca A = 5, B = 9, C = 12, D = 13, E = 16, F = 45 bulunur. Hoca kuyruğu her seferinde ağaç olarak çizmek yerine küçükten büyüğe sıralı gösterdi.

Adım	`extract_min` ile çekilenler	Yeni ara düğüm	Kuyruğun yeni hâli
1	A (5), B (9)	$5 + 9 = 14$	C12, D13, *14, E16, F45
2	C (12), D (13)	$12 + 13 = 25$	*14, E16, *25, F45
3	*14, E (16)	$14 + 16 = 30$	*25, *30, F45
4	*25, *30	$25 + 30 = 55$	F45, *55
5	F (45), *55	$45 + 55 = 100$	*100

Kuyrukta tek düğüm kalınca durulur. Min öncelikli kuyrukla başlanıp ekleme ve çıkarmalarla bir ağaç yapısı oluşturulmuştur. Kökteki ara düğümün değeri 100'dür:



Kökün sol çocuğu 45 frekanslı F'dir. Sağ çocuğu bir ara düğümdür (55). Onun solundaki ara düğümün (25) sol çocuğu 12 frekanslı C, sağ çocuğu 13 frekanslı D'dir. 55'in sağındaki ara düğümün (30) sağ çocuğu 16 frekanslı E, sol çocuğu bir ara düğümdür (14); onun çocukları 5 ve 9 frekanslı A ve B'dir.

8.5 Kodların Okunması

Sol çocuklara `0`, sağ çocuklara `1` kodu verilir. Kökten bir karaktere giden yol o karakterin kodudur.

- En sık karakter F (45) yalnız bir sola gidişle erişilir. Kodu tek bittir: `0`.
- `1`, `10`, `11` ara düğümlerdir; bunlarla karakter kodlanamaz.
- C `100`, D `101`, E `111` kodlarını alır; üçü de 3 bittir.
- A `1100`, B `1101` kodlarını alır; ikisi de 4 bittir.

Karakter	Frekans	Kod	Uzunluk
F	45	`0`	1
C	12	`100`	3
D	13	`101`	3
E	16	`111`	3
A	5	`1100`	4
B	9	`1101`	4

Hocanın istediği de buydu: F en kısa kodla, sonra E, D ve C daha uzun, en seyrek A ve B en uzun kodla kodlandı.

8.6 Kodlama Örneği ve Sıkıştırma Oranı

Hoca F çok geçtiği için içinde birkaç F olan rastgele bir metin seçti: `FDFEADFC` (8 karakter). Her karakter yeni koduyla yazılır:

Karakter	F	D	F	E	A	D	F	C
Kod	`0`	`101`	`0`	`111`	`1100`	`101`	`0`	`100`

Kodlanmış bit dizisi `010101111001010100`'dür ve uzunluğu 19 bittir. Hoca bunu normalde $4 \times 8 = 32$ bit ile kodlayacağını söyledi ve kazancı $32 - 19 = 13$ bit, oranı $13/32 \approx \%40$ olarak hesapladı.

> **Ders sonrası not:** Metinde 4 değil 8 karakter vardır (hoca da karakterleri sayarken 8 buldu). 8 bitlik sabit kodla $8 \times 8 = 64$ bit gerekir. 19 bit ile kazanç $64 - 19 = 45$ bit, yani yaklaşık $\%70$'tir ($45/64 \approx 0{,}70$). 6 sembol için 3 bitlik sabit uzunluklu kodla karşılaştırılırsa $8 \times 3 = 24$ bit gerekir; kazanç 5 bit, yaklaşık $\%21$'dir. 19 bitlik sonuç python ile doğrulandı.

8.7 Kod Çözme

Kodu çözmek için ağacın elde olması gerekir; bu yüzden ağaç karşı tarafa mutlaka verilmelidir. Bit dizisi soldan okunur ve kökten başlanarak ağaçta yürünür; bir yaprağa varınca karakter yazılır ve köke dönülür:

- `0` → F.
- `1`, `0` hâlâ ara düğüm; `1` gelince D (`101`).
- `0` → F.
- `111` → E.
- `1100` → A.
- `101` → D.
- `0` → F.
- `100` → C. Dizinin sonu geldiği için durulur.

Sonuç `FDFEADFC`'dir; aynı metin ağaç kullanılarak geri elde edilir.

> **Ek bilgi (derste anlatılmadı):** Hoca "Huffman kodlarının bir güzelliği de ara düğümler…" diye başlayıp cümleyi tamamlamadan çözmeye geçti. Bu güzellik önek (prefix) özelliğidir. Karakterler yalnız yapraklarda durduğu için hiçbir karakterin kodu başka bir karakterin kodunun başlangıcı olamaz. Bu yüzden kodlar arasında ayırıcı gerekmez ve bit dizisi tek bir biçimde çözülür.

Ağacı da göndermenin bir ek yükü (overhead) vardır. Bu küçük örnekte görece büyük görünür, ama kilobayt ya da megabayt mertebesinde yüzlerce, binlerce karakterlik verilerde ağacı eklemek toplam iletilen bilgiye çok az yük getirir.

Hoca bunun öncelikli kuyruğun gündelik hayattaki bir uygulaması olduğunu, zip gibi sıkıştırma araçlarında benzer bir yapının kullanıldığını söyledi.

> **Ek bilgi (derste anlatılmadı):** Zip dosyalarında yaygın olan DEFLATE yöntemi, büyük olasılıkla tekrar eden dizileri önce LZ77 ile kısaltır, sonra çıkan sembolleri Huffman kodlamasıyla kodlar. Yani Huffman tek başına değil, başka bir sıkıştırma adımıyla birlikte kullanılır.

Örnek Sorular

1. (derste sözlü, 00:37:42) Max öncelikli kuyruğa yeni eleman eklerken son göze neden önce $-\infty$ yazılır, sonra `increase_key` çağrılır?

> **Çözüm (derste verildi):** Heap düzenini bozmamak için. Eleman yerine taşınmadan önce bir `extract` işlemi araya girerse, $-\infty$ hiçbir ebeveynden büyük olmadığı için yapı geçerli bir heap olarak kalır. Doğrudan gerçek değer yazılsaydı heap düzeni bozuk kalabilirdi.

2. (derste sözlü, 01:04:12) 1'den n'e kadar indisli bir dizi verildi. Bu dizinin bir (max) heap olup olmadığını nasıl bulursunuz?

> **Çözüm (derste verildi):** İki kontrol yapılır. (1) `is_complete_binary_tree`: Bütün seviyeler dolu, son seviye soldan dolu mu? (2) `parent_control`: $i = \lfloor n/2 \rfloor$'den 1'e

kadar her i için $A[i] \geq A[2i]$ ve (varsa) $A[i] \geq A[2i+1]$ mi? İkisi de doğruysa `return true`. Örneğin `20, 15, 8, 10, 5, 7, 6, 2, 9, 1` bir max heap'tir; `2, 4, 8, 9, 7` ise max heap değildir (python ile doğrulandı).

3. (derste sözlü, 01:13:56) n farklı elemandan oluşan sırasız bir dizide, $k \leq n$ olmak üzere, k'ıncı en küçük elemanı bulan algoritmayı tasarlayınız ve karmaşıklığını veriniz.

> **Çözüm (derste verildi):** Düz çözüm: sırala, k'ıncı gözü döndür; seçmeli sıralamayla $O(n^2)$, heap sort ile $O(n \log n)$. Min heap ile: heap kur, k kez `extract_min` yap; hoca $O((n+k)\log n)$ verdi (`build_heap` ile $O(n + k \log n)$). En iyisi: ilk k elemandan max heap kur, kalan $n-k$ elemanı kökle karşılaştır, küçükse kökü çıkarıp elemanı ekle; sonunda kök cevaptır. Zaman $O(n \log k)$, ek alan $O(k)$.

4. (derste sözlü, 01:34:37) Bir min heap verildi. Bunu bir max heap'e nasıl dönüştürürsünüz? (Hoca tahtada `2, 4, 8, 9, 7` min heap'ini çizdi.)

> **Çözüm (derste verildi):** Son elemanı kök yapmak işe yaramaz; örnekte son eleman 7, en büyük ise 9'dur. Doğrusu heap kurar gibi $\lfloor n/2 \rfloor$'den 1'e kadar bütün ebeveynlere `max_heapify` uygulamaktır. Max heap'ten min heap'e geçmek için aynı döngüde `min_heapify` kullanılır.

> **Çözüm (derste verilmedi):** Hocanın çizdiği heap için ($n = 5$): $i = 2$'de 4'ün çocukları 9 ve 7'dir; 9 yukarı çıkar → `2, 9, 8, 4, 7`. $i = 1$'de 2'nin çocukları 9 ve 8'dir; 9 yukarı çıkar → `9, 2, 8, 4, 7`; 2 aşağıda yine heapify edilir, çocukları 4 ve 7'dir, 7 yukarı çıkar → `9, 7, 8, 4, 2`. Sonuç bir max heap'tir (python ile doğrulandı).

5. (derste sözlü, 02:04:24) A = 5, B = 9, C = 12, D = 13, E = 16, F = 45 frekanslarıyla kurulan Huffman ağacına göre `FDFEADFC` metni kaç bitle kodlanır? Sıkıştırma oranı nedir?

> **Çözüm (derste verildi):** Kodlar F `0`, C `100`, D `101`, E `111`, A `1100`, B `1101`'dir. Kodlanmış dizi `0101011111001010100`, uzunluğu 19 bittir. Hoca 32 bitle karşılaştırıp yaklaşık %40 kazanç buldu. Doğru karşılaştırma için 8. bölümdeki ders sonrası nota bakın: 8 bitlik kodla 64 bit gerekir ve kazanç yaklaşık %70'tir; 3 bitlik sabit kodla 24 bit gerekir ve kazanç yaklaşık %21'dir.

6. (derste sözlü, 02:05:30) `0101011111001010100` bit dizisini aynı Huffman ağacıyla çözünüz.

> **Çözüm (derste verildi):** Soldan okunup ağaçta yürünür; yaprağa varınca karakter yazılıp köke dönülür: `0` F, `101` D, `0` F, `111` E, `1100` A, `101` D, `0` F, `100` C. Sonuç `FDFEADFC`'dir.

Dersin Toparlaması

Bu derste öncelikli kuyruk veri yapısı, geçen haftanın heap konusunun bir uygulaması olarak ele alındı. Öncelikli kuyruk FIFO değildir; elemanlar anahtar değerlerine göre işlenir. Max öncelikli kuyruk max heap ile, min öncelikli kuyruk min heap ile gerçekleştirilir. En öncelikli eleman köktedir ve $O(1)$'de okunur; `extract`, `insert`, `increase_key` ve `decrease_key` $O(\log n)$'dir. İşletim sistemi işlem zamanlayıcısı max, olay güdümlü benzetici ise min öncelikli kuyruğa örnektir. Yeni eleman eklenirken önce $-\infty$ yazılması heap düzenini her an korur. Slaytlardaki C kodu, sözde kodun sınır kontrolleriyle C'ye çevrilmiş hâlidir.

Ardından üç heap sorusu üzerinde düşünüldü: Bir dizinin heap olup olmadığı (tamlık ve ebeveyn kontrolü), k'ıncı en küçük eleman (sıralama, min heap ve k elemanlı max heap ile $O(n \log k)$) ve min heap'i max heap'e çevirme (bütün ebeveynlere `max_heapify`). Hocanın vurguladığı ders şuydu: Bir problem için önce uygun veri yapısını seçmek, sonra o yapının varyantlarını düşünerek çözümü daha da iyileştirmek gerekir. Son olarak Huffman ağacı, min öncelikli kuyruğun gündelik bir uygulaması olarak anlatıldı: Frekanslar sayılır, en küçük iki düğüm tekrar tekrar birleştirilir, sol kenar `0` ve sağ kenar `1` alınarak sık karakterlere kısa kodlar verilir ve veri kayıpsız sıkıştırılır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.